

The production 3-command workflow: research, validation, execution to a merged PR

Overview

Plan-Validate-Execute is a production workflow in three commands that extends native Plan Mode: `/plan-start`, `/plan-validate`, `/plan-execute`. Each run feeds an ADR learning loop that reduces human interruptions over time. The guide describes it as Tier 2 production practice; install its custom commands before use.

The 3 commands

```
/plan-start    # PRD + dynamic research team + ADRs
/plan-validate # Structural checks + triggered specialist agents
/plan-execute # Worktree + TDD + parallel execution + merged PR
```

Run `/clear` between each command: every command reads its inputs from disk (the plan file, ADRs), nothing needs to stay in context memory.

/plan-start: 4-phase planning

Phase	Content
1. PRD & design	Missing/ambiguous requirements, compliance
2. Technical	Explore agents, ADR creation for new decisions
3. Scope	Research team selection (tier 0 to 4)
4. Research	Agents run in parallel, <code>planning-coordinator</code> (Opus) synthesizes

If 3+ confirmed ADRs already cover the decision, phase 2 auto-resolves with no agent.

/plan-validate: 2-layer validation

Layer 1 (structural, instant): plan format, task ordering, ADR consistency. Layer 2 (specialists, 0 to 8 agents by risk): `security-reviewer`, `db-migration-reviewer`, `performance-reviewer`, among others, each triggered by a precise criterion (auth, schema migration, new route...). Every issue must be resolved: auto-resolved from an existing ADR or `PATTERNS.md` entry, or settled by a human, which becomes a rule for next time.

/plan-execute: to a merged PR

Isolated worktree, TDD scaffolding, parallel execution of independent tasks (commit per task), quality gate (tests + smoke test), PR and squash merge, worktree cleanup. If the quality gate fails: up to 3 auto-fix attempts by dedicated debug agents, then human notification.

ADR learning loop

Every significant architectural decision generates an ADR. 1 ADR: watching. 2 ADRs: recommended default. 3+ ADRs: confirmed, auto-resolved on future plans, candidate for promotion to a hard rule in `CLAUDE.md`. The guide claims roughly 80% auto-resolution on a 20-plan project; no reproducible benchmark is supplied.

When to use this pipeline

Criterion	Answer
Multi-layer feature (API + DB + UI)	Yes
Sensitive change (auth, payments, PII)	Yes

Criterion	Answer
Type, trivial refactor	No, native <code>/plan</code> is enough
Exploratory prototyping with unclear requirements	No

Cost

Measure all planning, validation, implementation and retry calls. Current standard rates per million input/output tokens are \$2/\$10 for Sonnet 5.5 and \$4/\$20 for Opus 5.5; a task label alone does not determine the pipeline cost.

Auto plan mode

An `--append-system-prompt` request does not enforce native Plan Mode. Use permissions to restrict tools:

```
alias claude-safe='claude --append-system-prompt \
  "$(cat ~/.claude/auto-plan-mode.txt)'"
```

The `auto-plan-mode.txt` file instructs Claude to present a plan and wait for explicit approval before each tool. Useful for sessions on critical or unfamiliar codebases, outside the 3-command pipeline.